

ECOCIVIX AI

07 — Database & Data Model

TEAM VELOCITY • HackYatra × EDM 3.0 • PRE-HACKATHON PLANNING

Purpose: Define the relational data model required for authentication, issue reporting, AI analysis, assignments, timelines and community intelligence.

Status: PRE-HACKATHON DRAFT / PLANNING SPECIFICATION

Approval authority: Garv Nain

Core Entities

- users; issues; issue_ai_analysis; issue_updates; departments; assignments.
- Additional tables may be introduced only when a concrete product requirement exists.

Users

- Identity is handled through Supabase Auth; application profile/role data can be stored in a profile table or equivalent structure.
- Role must be explicitly stored and verified server-side.

Issues

- Core fields: id, reporter/user reference, title, description, status, priority, category, location fields where supported, image/storage reference where supported, timestamps.

AI Analysis

- Store structured AI assessment separately or as a clearly versioned relation: category, priority, summary, department, environmental impact/reason, safety risk/reason, suggested action, provider metadata if appropriate, timestamps.

Issue Updates

- Timeline events should capture issue reference, actor, event type, message/notes where appropriate, timestamp.

Departments

- Store department identifiers/names used for routing and assignment. Avoid hardcoding department lists across frontend code.

Assignments

- Link issue to responsible department and, where required, assigned administrator/operator. Track assignment time and status.

Relationships

- User 1→many Issues; Issue 1→1 or many AI analyses depending on versioning decision; Issue 1→many Updates; Department 1→many Assignments; Issue 1→many Assignments if reassignment is supported.

Indexes

- Index frequently filtered fields such as status, priority, category, created_at and department/assignment references. Exact indexes should be confirmed after query patterns are known.

Data Integrity

- Foreign keys, non-null constraints where required, enum/check constraints where practical, timestamps, and server-side validation.

Privacy

- Store only necessary personal data. Do not expose private user information in public community intelligence.

Community Analytics

- Derive counts and patterns from issues rather than storing fake aggregates. Use database queries/views or backend aggregation as appropriate.

Migration Strategy

- Schema changes should be tracked and reproducible. Do not manually alter production data during the final demo without recording the change.

Acceptance Criteria

- Schema supports core MVP flows, real persistence, issue lifecycle, AI analysis, admin actions, timeline and community metrics.